

RAPPORT DE PROJET : SIMULATION DE PROPAGATION DE FEU EN PLAN 2D

Membres de l'équipe : Babikian Benjamin, Bultelle Johan, Bagayoko Oumar,
Benyoucef Locqman

Enseignant Référent : Grignon Romuald

1. Description de l'équipe et du sujet

- Le Sujet

Ce projet consiste en la conception et le développement d'une application de simulation de propagation d'un incendie de forêt au sein d'un plan en deux dimensions. Reposant sur le principe des automates cellulaires, le système modélise une grille de cellules (composé d'arbres et de broussailles) dont les états évoluent de manière dynamique à chaque unité de temps. La simulation intègre des paramètres physiques et climatiques, tels que le taux d'humidité initial de la végétation, la charge de combustible disponible, ainsi que l'impact vectoriel du vent (vitesse et direction) sur la vitesse et la trajectoire du feu.

- L'Équipe

L'équipe est composée de quatre étudiants. Le groupe possède une bonne entente et a déjà travaillé ensemble sur d'autres projets scolaires au cours de l'année. Cette habitude de collaborer a grandement facilité la communication et la coordination tout au long du projet.

2. Organisation et flux de travail

Pour mener à bien les objectifs du cahier des charges, notre équipe a mis en place une méthodologie de travail agile et itérative, rythmée par un suivi académique rigoureux.

- Le rythme des réunions et suivi

Notre flux de travail s'est structuré autour de 6 réunions avec notre professeur référent, à raison d'une à deux sessions par semaine. Ces points de contrôle réguliers ont suivi une dynamique précise :

- Bilan d'avancement : Présentation à l'enseignant des fonctionnalités codées depuis la session précédente.
- Revue de code et corrections : Analyse critique par le professeur, corrections de nos modélisations algorithmiques et conseils d'optimisation.

- Cadrage : Validation des choix techniques avant d'attaquer les briques suivantes.
- Anticipation : Discussion autour des prochaines étapes et des différentes manières de surmonter les difficultés à venir.

- La répartition des tâches

À l'issue de chaque fin de réunion avec l'enseignant, la date de la rencontre suivante était fixée, matérialisant notre "DeadLine" interne. De plus, nous prenions des notes de ce qu'il restait à faire. Ainsi, l'équipe se réunissait alors pour planifier le "sprint" :

- Répartition par affinités et compétences : Les tâches étaient distribuées selon les préférences et les forces de chacun.
- Engagement individuel : Chaque membre s'engageait à livrer sa brique technique entièrement fonctionnelle et testée avant la réunion suivante, garantissant un livrable incrémental constant.
- Collaboration Git : L'utilisation d'un dépôt GitHub commun a permis de centraliser le code et de fusionner nos travaux de manière asynchrone.

3. Problèmes rencontrés et Solutions apportées

Problème : Différents IDE

- Description : Au départ, tous les membres de l'équipe n'avaient pas pour habitude de coder sur le même environnement de développement (IDE).
- Solution : Nous nous sommes mis d'accord pour utiliser VsCode et chacun s'est arrangé pour que ça fonctionne sur sa machine.

Problème : Synchronisation du code en équipe

- Description : Lors des premières phases, la fusion des fichiers modifiés par chacun en local provoquait des conflits majeurs, notamment sur le fichier pivot `Controller.java`.
- Solution : Mise en place d'une meilleure communication avant chaque git push et découpage du code en sous-fonctions isolées pour limiter les modifications simultanées sur les mêmes lignes de code.

Problème : Mauvaise organisation du code

- Description : On s'est rendu compte que notre Controller.java servait de bibliothèque de fonctions en plus d'être le fichier possédant la partie gestion de l'interface.
- Solution : On a créé un nouveau package « display » dans lequel on a créé un fichier pour assumer ce rôle de bibliothèque de fonctions. Cela nous permis de réorganiser les différents bouts de code

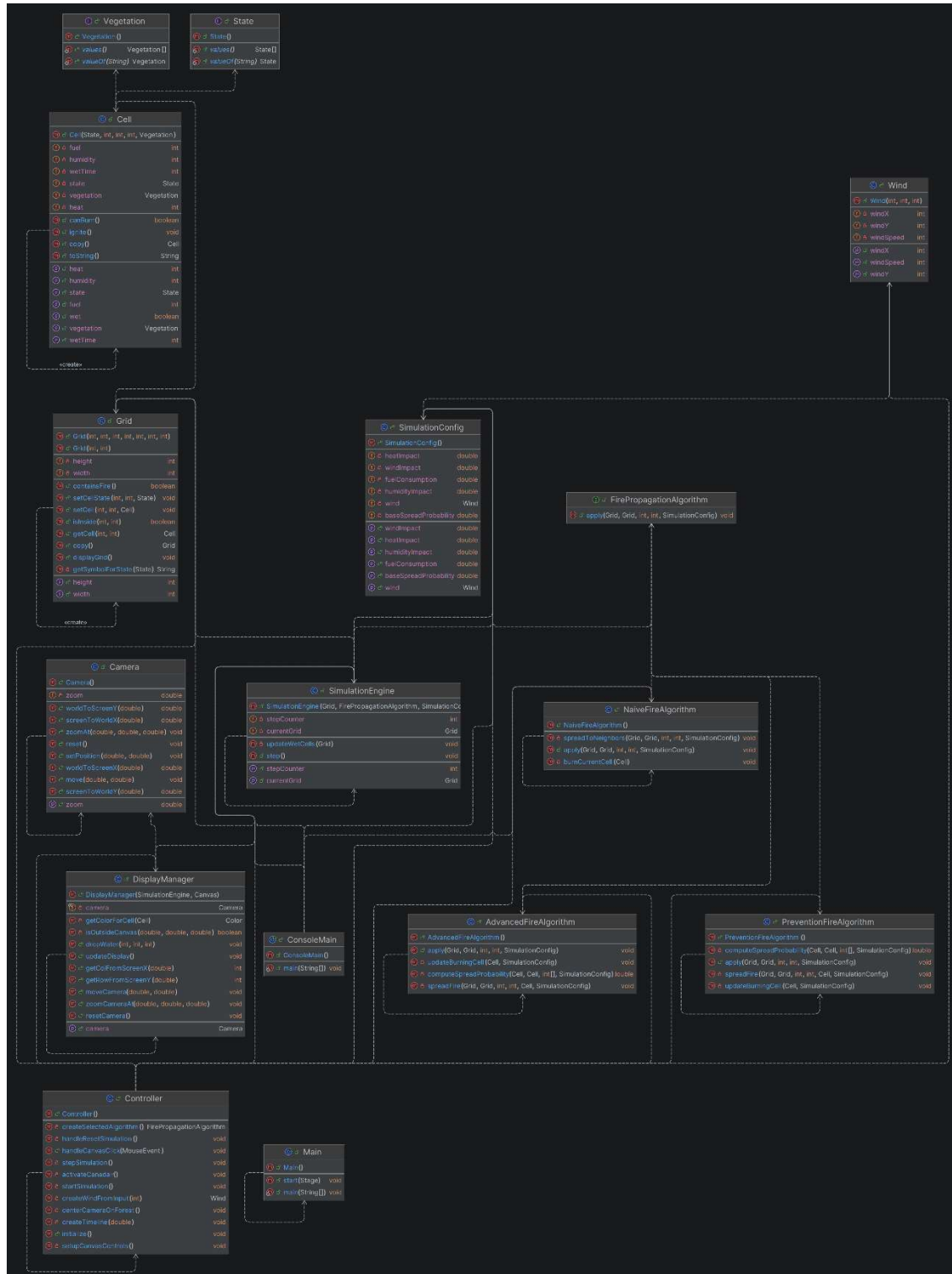
4. Résultats et Analyse

L'application finale répond strictement aux exigences du cahier des charges.
L'utilisateur dispose d'un outil complet capable de :

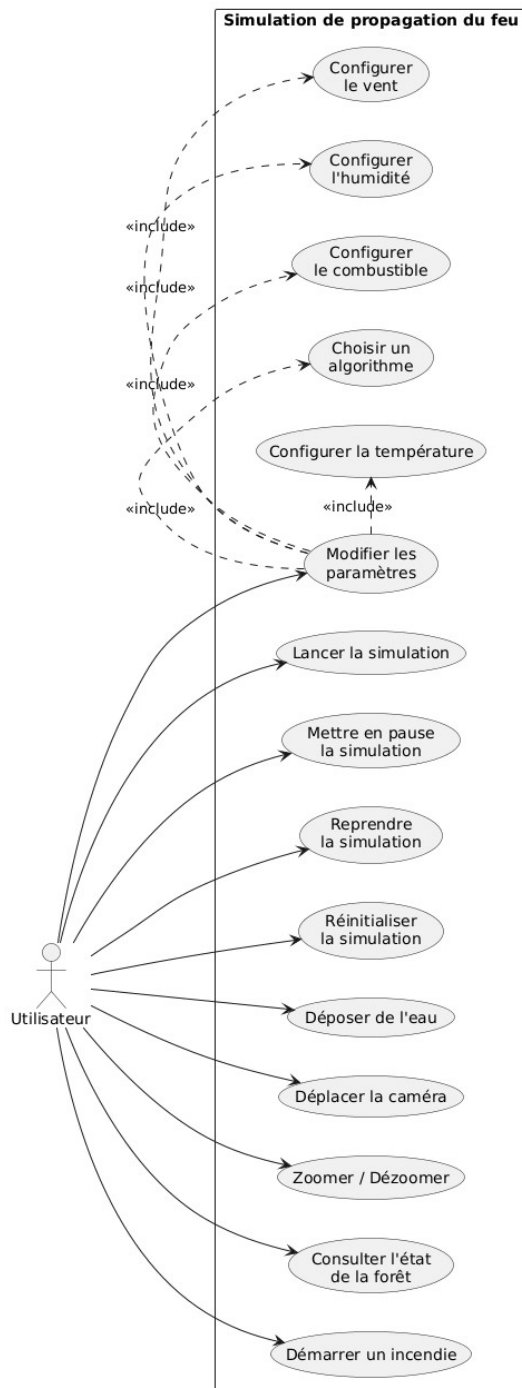
- Générer des forêts paramétrables au niveau des valeurs « météo » telles que l'humidité, la densité de combustible, la chaleur etc...
- Manipuler le temps (lancement fluide, pause, ou mode manuel "Step by Step", vitesse de la simulation ajustable).
- Interagir en temps réel (déplacement de caméra, extinction ciblée par largage d'eau via des canadais).

5. Annexes Techniques

- Document de conception UML (Diagramme de classes)



Document des Cas d'Utilisation (Use Case)



Code UML correspondant :

@startuml

left to right direction

actor Utilisateur

rectangle "Simulation de propagation du feu" {

usecase "Lancer la simulation" as UC1

usecase "Mettre en pause\nla simulation" as UC2

usecase "Reprendre\nla simulation" as UC3

usecase "Réinitialiser\nla simulation" as UC4

usecase "Modifier les\nparamètres" as UC5

usecase "Configurer\nle vent" as UC6

usecase "Configurer\nl'humidité" as UC7

usecase "Configurer\nle combustible" as UC8

usecase "Choisir un\nalgorithme" as UC9

usecase "Déposer de l'eau" as UC10

usecase "Déplacer la caméra" as UC11

usecase "Zoomer / Dézoomer" as UC12

usecase "Consulter l'état\nde la forêt" as UC13

usecase "Démarrer un incendie" as UC14

usecase "Configurer la température" as UC15

}

Utilisateur --> UC1

Utilisateur --> UC2

Utilisateur --> UC3

Utilisateur --> UC4

Utilisateur --> UC5

Utilisateur --> UC10

Utilisateur --> UC11

Utilisateur --> UC12

Utilisateur --> UC13

Utilisateur --> UC14

UC5 .> UC6 : <<include>>

UC5 .> UC7 : <<include>>

UC5 .> UC8 : <<include>>

UC5 .> UC9 : <<include>>

UC5 .> UC15 : <<include>>

@enduml